

## Week 5 Applied Class

### Version Control with Git: Collaboration & History

#### Overview

In this applied class, you will take a local Python project and use it in a collaborative remote workflow. You will connect a local repository to the FIT GitLab server, work with remotes from both the terminal and VS Code, invite another student to contribute, create and review a merge request, and practise history-analysis and recovery tasks.

**Important note.** Python is used here only as an example language because it is lightweight and easy to demonstrate in this applied class. The same Git collaboration ideas also apply to projects in other languages and toolchains.

**Important note about terminology.** GitLab uses the term *merge request (MR)* for the same general review workflow that GitHub calls a *pull request (PR)*. In this worksheet, we will use **MR** when referring to the GitLab interface and **PR-style workflow** when referring to the general idea.

#### Objectives

- To connect a local Git repository to a remote repository in FIT GitLab and to use VS Code and terminal workflows for fetch, pull, push, and branch publishing.
- To practise a branch-based team workflow in which one student publishes a feature branch, another student reviews it, and the branch is integrated through a merge request.
- To inspect repository evolution with `git log`, `git diff`, and `git blame`, and to use `git bisect`, `git revert`, and `git reset` in appropriate situations.
- To understand how rebase can update or tidy local feature-branch history, and why rebasing shared history must be handled carefully.

#### Working Mode for This Applied Class

**Work in pairs if possible.** Many of the collaboration tasks are easiest if one student acts as the project owner / maintainer and the other acts as a collaborator / developer.

##### Suggested roles.

- **Student A:** repository owner / maintainer for the shared GitLab project.
- **Student B:** collaborator who is invited to the project and contributes through a feature branch and merge request.

If you are working alone, you can still complete most tasks by using a second clone of your own repository in another folder.

## Task 0: Recover or Bootstrap a Local Repository (Optional, 5 minutes)

If you already have the Week 4 repository, you may reuse it. The expected path is:

```
~/fit2109/week4/logtool_git
```

If you do **not** have a usable local repository, create one now:

```
mkdir -p ~/fit2109/week5/logtool_git_remote/{app,tests}
cd ~/fit2109/week5/logtool_git_remote
```

```
cat > README.md <<'MD'
# FIT2109 Log Tool
```

A small Python example used to practise Git collaboration and history tools.  
MD

```
cat > app/stats.py <<'PY'
def count_levels(lines):
    counts = {}
    for line in lines:
        level = line.strip().split(maxsplit=1)[0]
        counts[level] = counts.get(level, 0) + 1
    return counts
PY
```

```
cat > app/main.py <<'PY'
from app.stats import count_levels
```

```
SAMPLE_LINES = [
    "INFO 100 Startup complete",
    "WARN 200 Disk space low",
    "ERROR 404 Page not found",
    "ERROR 500 Database unavailable",
]
```

```
def main():
    summary = count_levels(SAMPLE_LINES)
    for key in sorted(summary):
        print(f"{key}: {summary[key]}")
```

```
if __name__ == "__main__":
    main()
PY
```

```
cat > tests/test_stats.py <<'PY'
from app.stats import count_levels
```

```
def test_count_levels():
    counts = count_levels(["ERROR a", "ERROR b", "INFO c"])
    assert counts["ERROR"] == 2
    assert counts["INFO"] == 1
PY

git init
git add README.md app/main.py app/stats.py tests/test_stats.py
git commit -m "Add initial Python log tool example"
```

**Checkpoint.** Show your tutor that you have a local Git repository with at least one commit.

## Task 1: Sign in to FIT GitLab and Create the Remote Project (10 minutes)

**Step 1.1: Open FIT GitLab.** In a browser, go to:

<https://git.infotech.monash.edu/>

The sign-in page uses **Monash SAML Login**. Sign in with your Monash account.

**Step 1.2: Create a new group.** Use the GitLab interface to create a new group.

1. Go to **Groups** → **New Group**.
2. Name it as:

fit2109-<your-email-id>

3. Keep the Group **Private**.
4. Create the Group.

**Step 1.3: Create a blank project.**

1. Go to the Group you have just created.
2. Select **Create project**.
3. Select **Create blank project**.
4. Give it a name such as:

fit2109-week5-collab-<your-email-id>

5. Keep the project **Private**.
6. **Do not initialise the repository with a README** if you plan to push an existing local repository to it.
7. Create the project.

**Step 1.4: Copy the clone URL.** From the project home page, locate the **Code** button and copy the **Clone with HTTPS URL**. Keep it ready.

**Step 1.5: Create a personal access token (PAT) for HTTPS and the GitLab extension.** The FIT GitLab sign-in page uses Monash SAML Login. For Git over HTTPS on GitLab instances using SAML or 2FA, it is recommended to use a personal access token rather than a normal password.

In the GitLab UI:

1. Select your avatar → **Edit profile**.
2. In the left sidebar, select **Access** → **Personal access tokens**.
3. Click **Add new token**.
4. Give the token a clear name, an expiry date, and select at least the `api` scope.
5. Generate the token and **save it immediately**. GitLab will not show it again after you leave the page.

## Task 2: Connect the Local Repository to GitLab (Terminal First, Then VS Code) (10 minutes)

**Step 2.1: Open your local repository.** If you are reusing the Week 4 repository:

```
cd ~/fit2109/week4/logtool_git
```

Otherwise, use the repository you created in Task 0.

**Step 2.2: Add the remote from the terminal.** Replace `<HTTPS_URL_FROM_GITLAB>` with the HTTPS URL you copied from the project page.

```
git remote add origin <HTTPS_URL_FROM_GITLAB>
git remote -v
```

**Step 2.3: Push the current local main branch to GitLab.**

```
git push -u origin main
```

When Git prompts for credentials over HTTPS, use your GitLab username and paste the personal access token as the password.

**Checkpoint.** Refresh the GitLab project page in the browser. Can you now see your repository files online?

**Step 2.4: Open the repository in VS Code.**

1. Open the project folder in VS Code.
2. Open the **Source Control** view.
3. Confirm that VS Code detects the Git repository automatically.
4. Use the Source Control view to inspect the current branch and commit history.

```

Username for 'https://git.infotech.monash.edu': dennis.liu
Password for 'https://dennis.liu@git.infotech.monash.edu':
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 6 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (8/8), 1016 bytes | 40.00 KiB/s, done.
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: =====
remote:
remote:     If you need a refresher on how to use Git, please take a look at
remote:     these videos: https://bit.ly/fitgitvideos
remote:
remote: =====
remote:
To https://git.infotech.monash.edu/dennisliu/fit2019-week-collab-dennisliu.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

```

Figure 1: A Successful Push

**Step 2.5: Connect the official GitLab extension in VS Code.** This step is optional but recommended if you want GitLab-side features such as merge-request access inside VS Code.

1. In VS Code, install the official **GitLab** extension (the GitLab-maintained extension).
2. Open the Command Palette (Ctrl + Shift + P) and run:

```
GitLab: Authenticate
```

3. Enter the instance URL:

```
https://git.infotech.monash.edu
```

4. Choose **PAT** authentication.
5. Paste your PAT when prompted.
6. Open the GitLab sidebar and confirm that the current project is detected.

### Task 3: Practise Push, Fetch, Pull, and Upstream Tracking (10 minutes)

**Step 3.1: Make a small local change.** Add one short line to `README.md` such as:

```
This repository is used for Week 5 collaboration practice.
```

**Commit from either the terminal or VS Code.**

**Terminal route:**

```
git add README.md
git commit -m "Clarify README for Week 5 collaboration"
```

**VS Code route:**

1. In Source Control, hover over `README.md` and stage it with `+`.
2. Enter a commit message.
3. Commit the staged change.

**Step 3.2: Push using both perspectives.****Terminal:**

```
git push
```

**VS Code:** Use **Source Control** → **More Actions (...)** → **Push**, or the status-bar sync control if available.

**Step 3.3: Inspect upstream tracking.**

```
git branch -vv
```

**Questions.**

1. Which branch is your current local branch tracking?
2. Why does `git push` now work without repeating `origin main`?

**Task 4: Invite Another Student to Collaborate on the Same Project (5 minutes)**

**Student A: invite Student B.** In GitLab:

1. Open your project.
2. In the left sidebar, select **Manage** → **Members**.
3. Select **Invite members**.
4. Enter Student B's GitLab username.
5. Assign the **Developer** role.
6. Invite the student.

**Checkpoint.** Can Student B now see the project in GitLab after accepting the invitation?

**Important note.** The project creator usually has the strongest permissions on the project. Default branch protections may prevent Developers from pushing directly to `main`, which is useful for MR-based practice.

## Task 5: Collaborator Clone and Create a Feature Branch (10 minutes)

**Student B: clone the shared project.**

**Terminal route:**

```
cd ~/fit2109/week5
git clone <HTTPS_URL_FROM_GITLAB> partner_clone
cd partner_clone
git branch -a
git remote -v
```

**VS Code route:**

1. Open the Command Palette and run `Git: Clone`.
2. Paste the HTTPS URL of the shared GitLab project.
3. Choose a local folder and open the cloned repository in VS Code.

**Student B: create and publish a feature branch.** In the clone, create a feature branch and make a focused change.

```
git fetch origin
git switch main
git pull
git switch -c feature-readme-links
```

Add the following section to `README.md`:

```
## Development workflow
- create a feature branch
- make focused commits
- open a merge request
```

Then commit and publish the branch:

```
git add README.md
git commit -m "Add README notes on feature-branch workflow"
git push -u origin feature-readme-links
```

**VS Code alternative.** Use the branch status item in the lower-left corner or the Source Control view to create and switch branches. After committing, VS Code may prompt you to **Publish Branch**. Accept the prompt.

**Checkpoint.** Refresh GitLab in the browser. Can both students see the new branch on the remote repository?

## Task 6: Open and Review a Merge Request (15 minutes)

**Student B: create the merge request.** Open the new branch in GitLab and create an MR into `main`.

**GitLab UI route:**

1. Open the project in GitLab.
2. Select **Code** → **Merge requests** → **Create merge request**.
3. Set:
  - source branch = `feature-readme-links`
  - target branch = `main`
4. Give the MR a meaningful title and short description.
5. Create the MR.

**Student A: review the MR.**

1. Open the MR in GitLab. (You should see **Merge requests** on the side bar on the left-hand side)
2. Inspect the diff and the commit list.
3. Leave at least one review comment asking for a small improvement.

**Student B: revise the same branch.** Make one extra improvement requested by Student A. For example, add another README bullet or tidy wording.

```
git add README.md
git commit -m "Refine README workflow notes after review"
git push
```

**Observation.** The open MR should update automatically because it is tied to the same source branch.

**Student A: merge when ready.** Once the MR is acceptable, merge it into `main` from the GitLab UI.

## Task 7: Observe Remote-Tracking Branches with Fetch and Pull (10 minutes)

This task is designed to make `main` and `origin/main` feel different.

**Student A: after the MR is merged, do nothing else locally yet.**

**Student B: in the clone used for the feature branch, inspect the state.**

```
git branch -vv
git log --oneline --graph --decorate --all
```



**Questions before synchronising.**

1. Is Student B's local `main` already updated just because the MR was merged on GitLab?
2. What do you expect `origin/main` to mean at this moment?

**Step 7.1: Fetch only.**

```
git fetch origin
git branch -vv
git log --oneline --graph --decorate --all
```

**Discuss.**

1. Which reference moved after `git fetch`?
2. Did the checked-out local branch move yet?

**Step 7.2: Pull to integrate.**

```
git switch main
git pull
```

**VS Code route.** Use **Source Control** → **More Actions (...)** → **Pull** and compare the effect.

**Task 8: Create a Team Conflict and Resolve It (15 minutes)**

**Goal.** Create one conflict on purpose so that both students see the workflow in a shared repository.

**Step 8.1: Student A makes a change on `main`.** In Student A's repository, make sure the local repository is up-to-date.

```
git switch main
git pull
```

Edit the first heading in `README.md` to:

```
# FIT2109 Log Inspector
```

Then commit and push:

```
git add README.md
git commit -m "Rename project title to Log Inspector"
git push
```

**Step 8.2: Student B changes the same heading on a new feature branch.** In Student B's clone:

```
git fetch origin
git switch -c feature-title-update
```

Edit the same heading so that it instead becomes:

```
# FIT2109 Log Analysis Toolkit
```

Then commit:

```
git add README.md
git commit -m "Rename project title to Log Analysis Toolkit"
```

**Step 8.3: Update the feature branch from the remote main.**

```
git fetch origin
git merge origin/main
```

**Observe what happens.**

1. Run `git status`. What does Git report?
2. Open `README.md`. Can you see the conflict markers?
3. Why could Git not resolve the change automatically?

**Step 8.4: Resolve the conflict.** Choose one final title (or write a better combined one), delete the conflict markers, and complete the merge.

```
git add README.md
git commit
```

**Checkpoint.** Run:

```
git log --oneline --graph --decorate --all
```

Can you now see a non-linear history?

## **Task 9: Optional — Update a Branch with Rebase Instead of Merge (10 minutes)**

This task is recommended only for local branches that have not yet been shared broadly.

**Step 9.1: Create a scratch branch and make a small local commit.**

```
git switch main
git pull
git switch -c feature-rebase-demo
```

Add one small README bullet, then commit it:

```
git add README.md
git commit -m "Add local rebase demo note"
```

**Step 9.2: Simulate upstream movement.** Have your partner add and push one more small commit to main.

**Step 9.3: Rebase onto the newer origin/main.**

```
git fetch origin
git rebase origin/main
```

### Questions.

1. What does rebasing try to achieve conceptually?
2. Why can rebasing make a feature branch history appear more linear?
3. Why should you avoid rebasing shared history casually?

**If a conflict happens.** Use the incremental workflow:

```
# resolve the files

git add conflicted-file.py
git rebase --continue
# or git rebase --abort
```

## Task 10: Inspect History with `log`, `diff`, and `blame` (10 minutes)

Use the repository history you have created so far.

**Run the following commands.**

```
git log
git log --follow -- README.md
git diff
git diff --staged
git diff HEAD~1 HEAD
git blame app/main.py
```

### Questions.

1. What kind of question is each command best suited to answer?
2. Why is `git log -follow - README.md` different from plain `git log`?
3. What is the difference between `git diff` and `git diff -staged`?
4. What does `git blame` tell you that `git log` does not tell you directly?

## Task 11: Optional — Mini Lab with `git bisect` (10 minutes)

This task creates a small regression on a scratch branch so that you can use `git bisect`.

**Step 11.1: Create a bisect demo branch.**

```
git switch main
git pull
git switch -c bisect-demo
```

**Step 11.2: Make a known-good commit.** Add this helper script:

```
cat > run_check.py <<'PY'
from app.stats import count_levels

counts = count_levels(["ERROR a", "INFO b", "ERROR c"])
assert counts["ERROR"] == 2
assert counts["INFO"] == 1
print("ok")
PY

git add run_check.py
git commit -m "Add small regression-check script"
GOOD_HASH=$(git rev-parse HEAD)
echo "$GOOD_HASH"
```

**Step 11.3: Introduce a bad commit on purpose.** Edit `app/stats.py` so that it incorrectly counts every line as INFO, or otherwise break the logic. Then commit the broken version.

```
git add app/stats.py
git commit -m "Introduce counting bug for bisect demo"
```

**Step 11.4: Add one more harmless commit.** For example, update `README.md` and commit it. This makes the bad change less obvious from the tip alone.**Step 11.5: Run bisect.**

```
git bisect start
git bisect bad
git bisect good <GOOD_HASH>
# test the checked-out midpoint:
python3 run_check.py
# if the test passes: git bisect good
# if the test fails: git bisect bad
# repeat until Git identifies the first bad commit
git bisect reset
```

**After the culprit is found.** Practise the recommended repair workflow:

```
git show <BAD_HASH>
git switch main
git switch -c fix/regression-from-<BAD_HASH>
```

```
# fix the bug properly
git add app/stats.py run_check.py
git commit -m "Fix regression introduced by <BAD_HASH>"
```

**Questions.**

1. Why does `git bisect` depend on a clear test for good vs bad?
2. Why are smaller commits easier to debug with bisect?

**Task 12: Recovery with `revert` and `reset` (10 minutes)**

**Step 12.1: Revert a bad shared-style commit on a scratch branch.** Use one recent commit hash from your history and practise:

```
git revert <bad-commit>
```

**Questions.**

1. What kind of history does `git revert` create?
2. Why is revert usually safer than reset on commits that may already be shared?

**Step 12.2: Reset a local-only commit on another scratch branch.** Create one throwaway commit, then try:

```
git reset --mixed HEAD~1
```

Observe what happened to HEAD, the staging area, and the working tree.

**Optional dangerous command — use with care only on a scratch branch.**

```
git reset --hard HEAD~1
```

**Concept check.** Write one sentence each for:

- `git revert` is best when ...
- `git reset -mixed` is best when ...
- `git reset -hard` is risky because ...

**[Reflection]** Which Week 5 command or workflow gave you the clearest insight into *shared history* rather than just *local history*? Choose one of the following and explain why in 4 – 5 sentences: `fetch`, `pull`, `push`, `merge request`, `merge conflict`, `rebase`, `blame`, `bisect`, `revert`, or `reset`.